DOKUMENTASI PERANGKAT LUNAK

GUI Panel Kontrol Robot Feeder Akuatik

Pengembangan dengan Metode SDLC — Prototyping Model

Judul Skripsi	Pengembangan Robot Akuatik untuk Feeder Otomatis di Tambak
Sub-sistem	Transmitter — GUI Kendali (Raspberry Pi + NRF24L01)
Metode SDLC	Prototyping Model
Framework GUI	Python 3 + CustomTkinter
Tanggal	25 July 2026

1. Pendahuluan

1.1 Latar Belakang

Robot akuatik feeder otomatis telah berhasil beroperasi di air: mampu bergerak lurus dengan *heading-hold* berbasis MPU6050 dan berbelok melalui *differential thrust* dua motor brushless. Kendali sebelumnya dilakukan lewat antarmuka baris perintah (CLI) pada berkas **receiverArduino.py** yang mengharuskan operator mengetik dimensi tambak secara manual dan tidak menampilkan status sistem. Untuk meningkatkan kegunaan (*usability*) dan menyediakan pemantauan, dikembangkan antarmuka grafis (GUI) yang berjalan di Raspberry Pi.

1.2 Tujuan

- Menyediakan input dimensi tambak (panjang & lebar dalam meter) yang mudah.
- Menerima **titik awal robot**, **lebar lintasan**, dan **kecepatan**, lalu menerjemahkan tambak persegi menjadi **path/jalur** berupa daftar langkah "maju x m / belok y°" (pola zigzag) yang diikuti robot.
- Menyediakan **keypad numerik pada layar** (point-and-click) agar dapat dioperasikan tanpa keyboard fisik.
- Menampilkan **preview jalur** pada kanvas (peta tambak + arah tempuh) dan daftar langkah gerak sebelum misi dikirim.
- Menampilkan status koneksi modul NRF24 dan keterjangkauan robot secara nyata.
- Memberi kontrol manual untuk mematikan komponen yang relevan.
- Menampilkan telemetri (estimasi) kecepatan & RPM baling-baling secara realtime.
- Menampilkan data sensor MPU6050 (roll, pitch, yaw/heading) secara nyata via serial USB.
- Mencatat seluruh aktivitas dan mengekspornya sebagai berkas log (CSV/TXT).

1.3 Ruang Lingkup & Batasan

GUI berjalan di sisi **Transmitter** (Raspberry Pi) sebagai pengganti CLI. Pada iterasi ini firmware robot (ESP32, **Finalnew.ino**) turut **diperbarui**: protokol misi diperluas (menerima kecepatan, lebar lintasan, dan arah belok) dan pola belok diperbaiki agar sapuan menutupi seluruh tambak (lihat 4.5). Konsekuensi batasan teknis yang teridentifikasi pada tahap analisis:

- **Komunikasi satu arah.** Raspberry hanya mengirim; robot hanya menerima. Robot tidak mengirim telemetri balik.
- **Tidak ada sensor RPM/encoder** pada robot. Maka nilai RPM & kecepatan yang ditampilkan adalah **ESTIMASI** yang dihitung dari nilai PWM throttle, bukan hasil pengukuran.
- **Kontrol komponen individual** (mis. mematikan satu motor atau MPU) memerlukan protokol dua arah — di luar lingkup iterasi ini; ditandai sebagai indikator *monitor*.

> **Catatan:** Kejujuran teknis ini justru menjadi bahan analisis yang baik: keterbatasan menjadi 'kebutuhan iterasi berikutnya' — sesuai semangat Prototyping.

2. Metodologi: SDLC Prototyping Model

Dipilih **Prototyping Model** karena kebutuhan GUI belum sepenuhnya pasti di awal dan paling baik ditemukan melalui purwarupa yang dievaluasi bersama dosen pembimbing. Cocok untuk sistem gabungan perangkat keras + lunak yang berevolusi.

2.1 Alasan Pemilihan (dibanding metode lain)

Metode	Kesesuaian untuk proyek ini
Waterfall	Rapi didokumentasikan, tetapi kaku bila kebutuhan berubah.
V-Model	Kuat di pengujian, tetapi mengasumsikan kebutuhan sudah pasti.
Incremental	Baik, mirip Prototyping namun tiap increment harus 'final'.
Prototyping (dipilih)	**Kebutuhan GUI ditemukan lewat purwarupa & umpan balik; iteratif; cocok HW+SW.**

2.2 Tahapan Prototyping

Fase	Aktivitas pada proyek ini
1. Pengumpulan Kebutuhan	Wawancara kebutuhan operator: input P×L, status NRF, kontrol komponen, telemetri realtime, ekspor log.
2. Perancangan Cepat (Quick Design)	Rancang tata letak GUI & pemisahan arsitektur core/GUI; definisikan protokol paket (misi/PING/STOP).
3. Bangun Prototipe	Implementasi feeder_core.py (RadioLink, MissionEstimator, ActivityLogger) dan feeder_gui.py (CustomTkinter).
4. Evaluasi oleh Pengguna	Demo ke pembimbing memakai MODE SIMULASI (tanpa hardware), lalu uji nyata di Raspberry Pi + robot.
5. Perbaikan (Refinement)	Revisi berdasar umpan balik: iterasi-1 (label 'estimasi', LED status, penanganan input); iterasi-2 (perencanaan jalur, titik awal & kecepatan, keypad layar, preview jalur, perbaikan pola belok). Ulangi fase 3–5.
6. Rekayasa Produk & Pemeliharaan	Stabilkan, dokumentasikan, siapkan iterasi lanjutan (telemetri 2 arah).

2.3 Siklus Iterasi (Prototyping Loop)

```
Kebutuhan -> Quick Design -> Bangun Prototipe
^ |
| v
Perbaikan <---- Evaluasi Pengguna <---+
(ulangi hingga pengguna puas -> Produk Final)
```

3. Analisis Kebutuhan

3.1 Kebutuhan Fungsional

Kode	Kebutuhan	Status
RF-1	Input panjang & lebar tambak (meter) dan kirim misi AUTO	Terpenuhi
RF-2	Menampilkan status modul NRF & keterjangkauan robot (ACK)	Terpenuhi (real)
RF-3	Mematikan komponen yang dapat dikontrol (NRF TX, PING, Motor)	Terpenuhi
RF-4	Menampilkan RPM & kecepatan baling-baling realtime	Terpenuhi (estimasi)

Kode	Kebutuhan	Status
RF-5	Mencatat & mengekspor log aktivitas (CSV/TXT)	Terpenuhi
RF-6	Perintah STOP membatalkan misi berjalan	Terpenuhi (real)
RF-7	Menampilkan data MPU6050 (roll/pitch/yaw) via serial USB	Terpenuhi (real)
RF-8	Input titik awal robot (sudut tambak) & kecepatan & lebar lintasan	Terpenuhi
RF-9	Menerjemahkan path jadi daftar langkah "maju x / belok y°" & preview kanvas	Terpenuhi
RF-10	Keypad numerik pada layar (operasi tanpa keyboard fisik)	Terpenuhi
RF-11	Mengirim parameter misi ke robot (kecepatan, lebar lintasan, arah belok)	Terpenuhi (real)

3.2 Kebutuhan Non-Fungsional

- **Keterandalan:** heartbeat PING tiap 0,5 s menjaga failsafe robot (stop < 3 s bila sinyal hilang).
- **Portabilitas:** berjalan di Raspberry Pi (NRF real) & PC (auto-fallback simulasi).
- **Usability:** indikator warna (hijau/oranye/merah), label satuan jelas, serta **operasi penuh lewat sentuhan** (keypad di layar + preview jalur) tanpa keyboard/mouse.
- **Kejujuran data:** nilai estimasi diberi label eksplisit agar tidak menyesatkan.

4. Perancangan (Quick Design)

4.1 Arsitektur Perangkat Lunak

Menerapkan *Separation of Concerns*: lapisan presentasi (GUI) dipisah dari logika (core), sehingga mudah diuji dan dikembangkan.

Modul	Tanggung jawab
feeder_gui.py	Presentasi: input, keypad layar, kanvas preview jalur, tombol, gauge, LED, tampilan log.
feeder_core.py -> plan_coverage / CoveragePlan	Terjemahkan tambak + titik awal + lebar lintasan -> daftar segmen "maju/belok" + encode_mission (paket ke robot).
feeder_core.py -> RadioLink	Abstraksi NRF24 (pyrf24) + auto-fallback MockRadio (simulasi).
feeder_core.py -> MissionEstimator	Cermin timeline zigzag firmware -> estimasi fase/PWM/RPM per waktu.
feeder_core.py -> ActivityLogger	Pencatatan aktivitas berstempel waktu + ekspor CSV/TXT.

4.2 Protokol Komunikasi (diperluas, tetap kompatibel-mundur)

Paket	Arah	Makna
P=<m>;L=<m>;MODE=AUTO; SPD=<m/s>;LW=<m>;DIR=<R|L>	Pi -> Robot	Perintah misi zigzag + kecepatan (SPD), lebar lintasan (LW), arah belok pertama (DIR). Field SPD/LW/DIR opsional; bila tak ada robot memakai default.

Paket	Arah	Makna
PING	Pi -> Robot	Heartbeat (jaga failsafe) tiap 0,5 s
STOP	Pi -> Robot	Batalkan misi / matikan motor
(ACK otomatis)	Robot -> Pi	Auto-ACK NRF; True = robot hidup & terjangkau

Paket lama **P=..;L=..;MODE=AUTO** tetap valid — firmware baru mengabaikan field yang tak ada dan memakai konstanta default, sehingga tidak memutus kompatibilitas.

4.3 Dasar Estimasi Telemetri

Karena tak ada sensor, GUI menghitung PWM yang *seharusnya* dijalankan robot pada detik ke-t (mengikuti pola gerak firmware), lalu mengonversinya:

```
throttle = (PWM - 1000) / (2000 - 1000)
RPM_est = throttle * MOTOR_KV * BATTERY_V (default KV=2200, V=11.1)
speed_est= ROBOT_SPEED_MPS * throttle / throttle(CRUISE_PWM)
```

> **Catatan:** Konstanta MOTOR_KV, BATTERY_V, CRUISE_PWM, TURN_90_MS wajib disamakan dengan firmware & dikalibrasi pada pengujian air agar estimasi mendekati kenyataan.

4.4 Integrasi Data MPU6050 (Serial USB)

Berbeda dengan telemetri gerak yang berupa estimasi, data orientasi diambil **langsung dari sensor MPU6050**. Sketsa **mpu.ino** (Test Part/MPU Test) pada ESP32 mencetak orientasi ke Serial tiap 200 ms; GUI membacanya melalui **pyserial** pada jalur USB yang **terpisah dari NRF**. Kelas **MPUSerialReader** mem-parse baris berikut:

```
Angle -> X: <roll> Y: <pitch> Z(Yaw): <yaw> | Gyro -> X: <gx> Y: <gy> Z: <gz>
```

Roll (X), Pitch (Y), dan Yaw/heading (Z) ditampilkan realtime dan dapat direkam ke log (1 Hz) untuk diekspor. Jalur serial cocok untuk fase **kalibrasi/uji bench** saat board tertambat kabel; saat robot berlayar bebas, data MPU realtime memerlukan telemetri NRF dua arah (iterasi berikutnya).

4.5 Perencanaan Jalur (Path Planning)

Fungsi **plan_coverage()** menerjemahkan tambak persegi menjadi jalur sapuan *boustrophedon* (zigzag). Dari masukan pengguna — panjang, lebar, **lebar lintasan**, **kecepatan**, dan **titik awal** (sudut tambak) — dihitung titik-titik belok (waypoint), lalu diturunkan menjadi daftar langkah eksplisit "maju x meter" dan "belok kanan/kiri y°". Daftar ini ditampilkan ke pengguna dan dasar paket yang dikirim ke robot (SPD/LW/DIR).

Koordinat: sumbu-x menyusuri lebar, sumbu-y menyusuri panjang. Robot menjelajah lurus arah-y tiap lintasan, lalu U-turn menggeser sejauh satu lebar lintasan arah-x.

```
Contoh: tambak 10 x 4 m, lebar lintasan 1 m, mulai Kiri-Bawah -> 4 lintasan:
1. Maju 10.0 m 5. Maju 10.0 m 9. Maju 10.0 m
2. Belok kanan 90 6. Belok kiri 90 10. Belok kanan 90
3. Maju 1.0 m 7. Maju 1.0 m 11. Maju 1.0 m
4. Belok kanan 90 8. Belok kiri 90 12. Belok kanan 90
13. Maju 10.0 m
Paket: P=10;L=4;MODE=AUTO;SPD=0.5;LW=1;DIR=R
```

Perbaikan pola belok (temuan iterasi ini). Versi sebelumnya melakukan U-turn yang **selalu belok kanan**. Secara geometri hal ini menggeser robot ke kanan lalu ke kiri bergantian, sehingga robot hanya bolak-balik di dua kolom dan **tidak menyapu seluruh lebar tambak**. Diperbaiki menjadi arah

U-turn **berselang-seling** (kanan-kanan, lalu kiri-kiri, dst) sehingga geseran selalu ke sisi yang sama — sapuan menutupi seluruh tambak. GUI dan firmware kini menghitung arah belok dari sumber yang sama, jadi **preview jalur = gerak nyata robot**.

> **Catatan:** Titik awal dapat dipilih lewat menu atau **diklik langsung pada kanvas peta**; arah belok pertama (DIR) dihitung dari titik awal agar robot menyapu ke arah dalam tambak.

5. Implementasi Prototipe

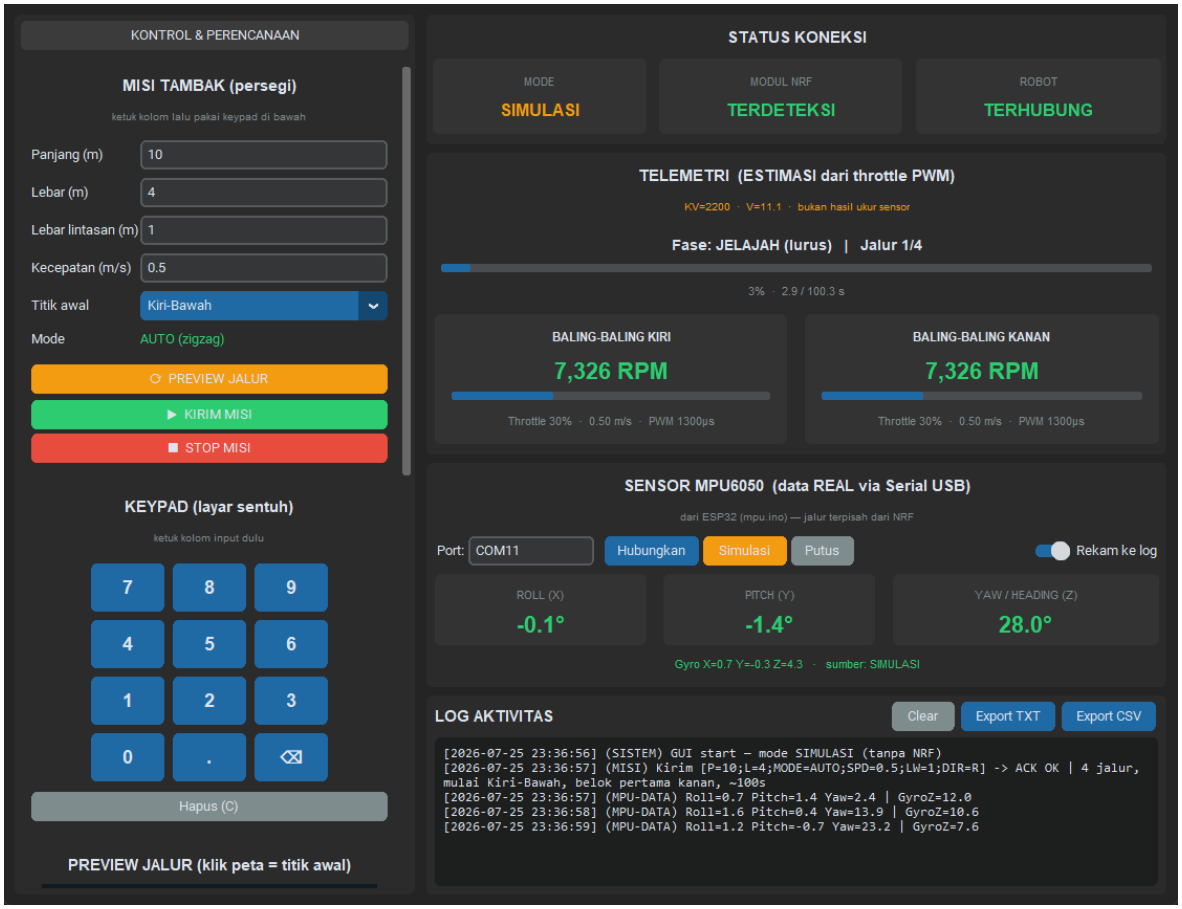
5.1 Fitur GUI

- **Panel Misi:** input Panjang, Lebar, Lebar lintasan, Kecepatan (m/s), dan Titik awal (sudut tambak); tombol PREVIEW JALUR, KIRIM MISI & STOP.
- **Keypad numerik:** tombol 0–9, titik, hapus & backspace pada layar — ketuk kolom lalu ketik; memungkinkan operasi **tanpa keyboard fisik** (layar sentuh).
- **Preview Jalur:** kanvas peta tambak menggambar lintasan zigzag berpanah + titik START (klik peta = pilih titik awal), disertai daftar langkah "maju/belok" dan ringkasan (jumlah lintasan, jarak, estimasi waktu).
- **Status Koneksi:** mode (REAL/SIMULASI), deteksi modul NRF, status robot.
- **Kontrol Komponen:** saklar NRF TX, Heartbeat PING, Motor (efek nyata lewat STOP); indikator monitor Motor Kiri/Kanan & MPU.
- **Telemetri:** fase misi, progress bar, gauge RPM/throttle/speed L & R.
- **Sensor MPU6050:** pilih port serial, tampil Roll/Pitch/Yaw realtime, opsi rekam ke log.
- **Log:** tampilan realtime + tombol Export CSV / Export TXT / Clear.

5.2 Mode Simulasi

Bila pyrf24 tidak tersedia atau NRF tak terdeteksi, RadioLink otomatis memakai MockRadio; demikian pula MPUSerialReader menyediakan mode simulasi bila serial tidak tersedia. Dengan begitu GUI tetap dapat didemokan dan diuji tanpa perangkat keras — sangat membantu pada fase evaluasi Prototyping.

5.3 Tampilan Antarmuka



Gambar 1. Antarmuka GUI: panel misi (dimensi, lebar lintasan, kecepatan, titik awal), keypad numerik & preview jalur di kolom kiri; status, telemetri, panel MPU6050 & log aktivitas di kolom kanan.

6. Pengujian (Black-Box)

Uji	Skenario	Hasil Diharapkan
T-1	Input P=10 L=4, klik KIRIM MISI	Paket terkirim, 4 jalur terhitung, gauge bergerak
T-2	Robot online (ACK) di simulasi	Status ROBOT = TERHUBUNG (hijau)
T-3	Robot offline	Status ROBOT = TERPUTUS (merah) < 3 s
T-4	Matikan saklar Motor	STOP terkirim, misi berhenti, LED motor merah
T-5	Input non-angka	Muncul dialog error, misi tidak dikirim
T-6	Klik Export CSV/TXT	Berkas log tersimpan & berisi seluruh event
T-7	Timeline estimasi habis	Fase = SELESAI, misi & heartbeat berhenti otomatis
T-8	Hubungkan MPU (serial/simulasi) + Rekam ke log	Roll/Pitch/Yaw tampil realtime, data MPU masuk log & ekspor
T-9	Isi dimensi/lebar lintasan/titik awal lalu Preview	Kanvas & daftar langkah "maju/belok" sesuai & berubah realtime
T-10	Ketuk kolom input lalu tekan keypad angka	Nilai kolom terisi tanpa keyboard fisik; preview ikut ter-update

Uji	Skenario	Hasil Diharapkan
T-11	Ganti titik awal (menu / klik kanvas)	Jalur & arah belok pertama (DIR) berubah sesuai sudut awal
T-12	Kirim misi dgn kecepatan berbeda	Paket memuat SPD/LW/DIR; estimasi waktu menyesuaikan kecepatan

7. Panduan Instalasi & Penggunaan

7.1 Instalasi

```
# Di Raspberry Pi (kendali nyata):
pip install customtkinter pyrf24

# Di PC (mode simulasi, tanpa NRF):
pip install customtkinter
```

7.2 Menjalankan

Cukup jalankan satu berkas entry point **receiverArduino.py**; GUI, telemetri, dan logging berjalan otomatis:

```
cd Master Code/Transmitter
python receiverArduino.py # buka GUI (default)
python receiverArduino.py --cli # mode teks lama (opsional)
```

7.3 Berkas Terkait

Berkas	Keterangan
receiverArduino.py	ENTRY POINT — jalankan ini -> GUI (opsi --cli utk mode teks lama)
feeder_gui.py	Aplikasi GUI (CustomTkinter)
feeder_core.py	Backend: RadioLink, MissionEstimator, ActivityLogger, MPUSerialReader
capture_gui.py	Skrip pengambil screenshot GUI untuk dokumentasi
build_docs_pdf.py	Generator dokumen PDF ini
Robot/New/Finalnew/Finalnew.ino	Firmware robot (ESP32) — diperbarui: parse SPD/LW/DIR + belok berselang-seling
Test Part/MPU Test/mpu/mpu.ino	Sketsa sumber data MPU6050 via serial

7.4 Deploy di Raspberry Pi (rekomendasi)

GUI dijalankan di Raspberry Pi karena memiliki GPIO/SPI untuk modul NRF24. GUI membutuhkan sesi **Desktop** (monitor HDMI atau VNC) — tidak bisa lewat SSH polos. Wiring NRF24L01 mengikuti pemetaan pin berikut (kode: RF24(24, 0) -> CE=GPIO24, CSN=CE0):

NRF24L01	Raspberry Pi (BCM)	Pin fisik
VCC (3.3V)	3.3V	1 / 17
GND	GND	6, 9, ...

NRF24L01	Raspberry Pi (BCM)	Pin fisik
CE	GPIO24	18
CSN	GPIO8 (CE0)	24
SCK	GPIO11	23
MOSI	GPIO10	19
MISO	GPIO9	21

Catatan: VCC NRF wajib 3.3V (bukan 5V); tambahkan kapasitor 10µF antara VCC–GND modul agar catu daya stabil.

Instalasi otomatis — ekstrak paket **raspberry_feeder.zip**, lalu jalankan skrip setup sekali:

```
cd ~/feeder # lokasi hasil ekstrak
chmod +x setup_pi.sh run.sh
./setup_pi.sh # aktifkan SPI, apt, venv, pip install
```

Instalasi manual (bila ingin bertahap):

```
sudo raspi-config nonint do_spi 0 # aktifkan SPI
sudo apt install -y python3-venv python3-tk
sudo usermod -aG spi,gpio,dialout $USER # lalu logout-login
python3 -m venv --system-site-packages ~/feeder-venv
source ~/feeder-venv/bin/activate
pip install -r requirements.txt
```

Menjalankan:

```
./run.sh # buka GUI
./run.sh --cli # mode teks lama
```

Catatan: Gunakan venv dengan --system-site-packages agar python3-tk (dari apt) terlihat di dalam venv; tanpa itu GUI gagal import tkinter. Bila status MODE tetap SIMULASI di Pi, berarti SPI belum aktif / wiring NRF salah (cek: ls /dev/spidev*).

8. Kesimpulan & Iterasi Berikutnya

Prototipe GUI berhasil memenuhi seluruh kebutuhan fungsional sesuai lingkup iterasi ini, dengan pemisahan arsitektur yang bersih dan mode simulasi yang mempermudah evaluasi. **Iterasi ini** — menindaklanjuti umpan balik pembimbing — menambahkan **perencanaan jalur** (dimensi + titik awal + lebar lintasan + kecepatan -> daftar langkah maju/belok), **keypad layar** untuk operasi tanpa keyboard, **preview jalur** point-and-click, protokol misi diperluas, serta **perbaikan pola belok** agar sapuan menutupi seluruh tambak. Sesuai Prototyping Model, keterbatasan yang tersisa menjadi rencana iterasi selanjutnya:

- Menambah **telemetri dua arah** (ACK payload NRF) agar status komponen & throttle asli dikirim balik dari robot.
- Memasang **sensor RPM (hall/encoder)** untuk mengganti estimasi dengan data ukur.
- Menambah perintah firmware untuk **kontrol komponen individual** (motor kiri/kanan, MPU).
- Belok berbasis **sudut yaw MPU** (bukan durasi), agar belokan presisi tanpa kalibrasi TURN_90_MS yang rawan.
- Mendukung **path waypoint bebas** (klik banyak titik) selain sapuan persegi.
- Kalibrasi konstanta (ROBOT_SPEED_MPS, TURN_90_MS, KV) pada pengujian air.